

lab3

```
1  function lab3
2      clear; clc; close all;
3      more off;
4
5      % =====
6      % Lab 3 - M/M/1/10 Simulation
7      % Questions (1) and (2)
8      % Updated version:
9      % 1. Plots ergodic probabilities with and without transient
10     %     on the same diagram
11     % 2. Places legends outside the plotting area
12     %
13     % File name must be: lab3.m
14     % Run with: lab3
15     % =====
16
17     SEED = 1;          % same seed in all runs
18     mu = 5;           % service rate (customers / min)
19     K = 10;           % system capacity
20     lambdas = [1, 5, 10];
21
22     % Required (T, B) pairs
23     configs = [
24         100000,    5000;
25         1000000,   20000;
26         10000000,  120000
27     ];
28
29     % -----
30     % DEBUG SECTION
31     % -----
32     % For question (1), keep DEBUG_TRACE = true to print the first
33     % 30 transitions. After verifying correctness, set it to false.
34     DEBUG_TRACE = true;
35     DEBUG_STEPS = 30;
36     TRACE_LAMBDA = 5;
37     TRACE_T = DEBUG_STEPS;
38     TRACE_B = 0;
39
40     % Sampling step for convergence plots of L
41     SAMPLE_EVERY = 1000;
42
43     fprintf('=====\\n');
44     fprintf('DEBUG TRACE - first %d transitions\\n', DEBUG_STEPS);
45     fprintf('lambda = %g, T = %d, B = %d, seed = %d\\n', ...
46         TRACE_LAMBDA, TRACE_T, TRACE_B, SEED);
47     fprintf('=====\\n');
48
49     simulate_mm1k_lab3(TRACE_LAMBDA, mu, K, TRACE_T, TRACE_B, ...
50         SEED, DEBUG_TRACE, DEBUG_STEPS, SAMPLE_EVERY);
```

```

51
52 fprintf('\nEnd of debug trace.\n');
53 fprintf('To disable debugging for the normal executions, set:\n');
54 fprintf('DEBUG_TRACE = false\n\n');
55
56 % -----
57 % NORMAL EXECUTIONS
58 % -----
59 results = cell(rows(configs), length(lambdas));
60
61 for cfg = 1:rows(configs)
62     T = configs(cfg, 1);
63     B = configs(cfg, 2);
64
65     for li = 1:length(lambdas)
66         lambda = lambdas(li);
67
68         fprintf('=====\n');
69         fprintf('Running simulation for lambda = %g, T = %d, B = %d\n', ...
70             lambda, T, B);
71         fprintf('=====\n');
72
73         results{cfg, li} = simulate_mm1k_lab3(lambda, mu, K, T, B, ...
74             SEED, false, DEBUG_STEPS, SAMPLE_EVERY);
75
76         print_results(results{cfg, li});
77     endfor
78
79     % -----
80     % Figure 1 for this (T,B): ergodic probabilities
81     % WITH and WITHOUT transient on the same diagram
82     % -----
83     figure(cfg);
84     clf;
85
86     for li = 1:length(lambdas)
87         subplot(length(lambdas), 1, li);
88
89         prob_matrix = [results{cfg, li}.P(:), results{cfg, li}.P_B(:)];
90         bar(0:K, prob_matrix, 'grouped');
91         grid on;
92
93         xlim([-0.5, K + 0.5]);
94         ylim([0, max(0.01, 1.10 * max(prob_matrix(:)))]);
95
96         title(sprintf(['Ergodic state probabilities ', ...
97             '(\lambda = %g, T = %d, B = %d)'], ...
98             lambdas(li), T, B));
99         xlabel('State');
100        ylabel('Probability');
101
102        hleg = legend('All transitions', 'After transient');
103        safe_legend_outside(hleg);
104    endfor

```

```

105
106 % -----
107 % Figure 2 for this (T,B): evolution of estimated mean
108 % number in system as a function of transitions
109 % -----
110 figure(cfg + rows(configs));
111 clf;
112
113 for li = 1:length(lambdas)
114     subplot(length(lambdas), 1, li);
115
116     plot(results{cfg, li}.sample_steps, ...
117          results{cfg, li}.L_evolution_all, 'r', 'LineWidth', 1.2);
118     hold on;
119
120     if ~isempty(results{cfg, li}.sample_steps_B)
121         plot(results{cfg, li}.sample_steps_B, ...
122              results{cfg, li}.L_evolution_B, 'b', 'LineWidth', 1.2);
123     endif
124
125     yl = ylim();
126     line([B, B], yl, 'Color', 'k', 'LineStyle', '--', 'LineWidth', 1.0);
127
128     grid on;
129     title(sprintf(['Evolution of estimated mean number in system ', ...
130                  '(\lambda = %g, T = %d, B = %d)'], ...
131            lambdas(li), T, B));
132     xlabel('Number of transitions');
133     ylabel('L estimate');
134
135     hleg = legend('All transitions', 'After transient', 'B cutoff');
136     safe_legend_outside(hleg);
137
138     hold off;
139 endfor
140 endfor
141 endfunction
142
143
144 function results = simulate_mm1k_lab3(lambda, mu, K, T, B, seed, ...
145                                     debug_trace, debug_steps, sample_every)
146
147     t_start = tic;
148
149     % Reset seed in every run, as required
150     rand("seed", seed);
151
152     % System state
153     state = 0;
154     current_time = 0.0;
155
156     % FIFO storage of accepted customers' arrival times
157     arrival_times = [];
158     arrival_after_B = [];

```

```

159
160 % -----
161 % Statistics - all transitions
162 % -----
163 arrival_attempts = 0;
164 rejections = 0;
165 time_in_state = zeros(1, K + 1);
166 total_time = 0.0;
167 area_num = 0.0;          % integral of N(t) dt
168 sojourn_sum = 0.0;
169 served_customers = 0;
170
171 % -----
172 % Statistics - after removing first B
173 % -----
174 arrival_attempts_B = 0;
175 rejections_B = 0;
176 time_in_state_B = zeros(1, K + 1);
177 total_time_B = 0.0;
178 area_num_B = 0.0;
179 sojourn_sum_B = 0.0;
180 served_customers_B = 0;
181
182 % -----
183 % For convergence / evolution L
184 % -----
185 n_samples = ceil(T / sample_every);
186
187 sample_steps = zeros(1, n_samples);
188 L_evolution_all = zeros(1, n_samples);
189
190 sample_steps_B = zeros(1, n_samples);
191 L_evolution_B = zeros(1, n_samples);
192
193 sample_idx = 0;
194 sample_idx_B = 0;
195
196 % -----
197 % Main simulation loop
198 % -----
199 for step = 1:T
200
201     % Event selection and total rate
202     if state == 0
203         is_arrival = 1;
204         total_rate = lambda;
205     else
206         total_rate = lambda + mu;
207         is_arrival = (rand() < lambda / (lambda + mu));
208     endif
209
210     % Time to next event ~ Exp(total_rate)
211     u = rand();
212     while u == 0

```

```

213     u = rand();
214 endwhile
215 dt = -log(u) / total_rate;
216 next_time = current_time + dt;
217
218 % State remains constant during [current_time, next_time)
219 time_in_state(state + 1) = time_in_state(state + 1) + dt;
220 total_time = total_time + dt;
221 area_num = area_num + state * dt;
222
223 if step > B
224     time_in_state_B(state + 1) = time_in_state_B(state + 1) + dt;
225     total_time_B = total_time_B + dt;
226     area_num_B = area_num_B + state * dt;
227 endif
228
229 % Process the event at next_time
230 if is_arrival
231
232     arrival_attempts = arrival_attempts + 1;
233     if step > B
234         arrival_attempts_B = arrival_attempts_B + 1;
235     endif
236
237     if state < K
238         % Accepted arrival
239         state_new = state + 1;
240
241         arrival_times(end + 1) = next_time;
242         arrival_after_B(end + 1) = (step > B);
243
244         accepted_text = 'accepted';
245     else
246         % Rejected arrival
247         state_new = state;
248         rejections = rejections + 1;
249
250         if step > B
251             rejections_B = rejections_B + 1;
252         endif
253
254         accepted_text = 'rejected';
255     endif
256
257 else
258     % Departure
259     state_new = state - 1;
260
261     customer_arrival_time = arrival_times(1);
262     customer_after_B = arrival_after_B(1);
263
264     arrival_times(1) = [];
265     arrival_after_B(1) = [];
266

```

```

267     sojourn = next_time - customer_arrival_time;
268
269     sojourn_sum = sojourn_sum + sojourn;
270     served_customers = served_customers + 1;
271
272     % For post-transient W, keep only customers accepted after B
273     if customer_after_B
274         sojourn_sum_B = sojourn_sum_B + sojourn;
275         served_customers_B = served_customers_B + 1;
276     endif
277 endif
278
279 % Debug print for the first 30 transitions
280 if debug_trace && step <= debug_steps
281     if is_arrival
282         printf(['Step %2d | state=%2d | event=arrival (%s) | ', ...
283             'arrivals=%2d | new state=%2d\n'], ...
284             step, state, accepted_text, arrival_attempts, state_new);
285     else
286         printf(['Step %2d | state=%2d | event=departure      | ', ...
287             'arrivals=%2d | new state=%2d\n'], ...
288             step, state, arrival_attempts, state_new);
289     endif
290 endif
291
292 % Move to next state / time
293 state = state_new;
294 current_time = next_time;
295
296 % Sample convergence curve of L
297 if mod(step, sample_every) == 0 || step == T
298     sample_idx = sample_idx + 1;
299     sample_steps(sample_idx) = step;
300     L_evolution_all(sample_idx) = area_num / total_time;
301
302     if total_time_B > 0
303         sample_idx_B = sample_idx_B + 1;
304         sample_steps_B(sample_idx_B) = step;
305         L_evolution_B(sample_idx_B) = area_num_B / total_time_B;
306     endif
307 endif
308 endfor
309
310 % -----
311 % Final metrics
312 % -----
313 results.lambda = lambda;
314 results.mu = mu;
315 results.K = K;
316 results.T = T;
317 results.B = B;
318 results.runtime_sec = toc(t_start);
319
320 % All transitions

```

```

321     results.P = time_in_state / total_time;
322     results.L = area_num / total_time;
323     results.P_rej = rejections / arrival_attempts;
324
325     if served_customers > 0
326         results.W = sojourn_sum / served_customers;
327     else
328         results.W = NaN;
329     endif
330
331     % After transient
332     if total_time_B > 0
333         results.P_B = time_in_state_B / total_time_B;
334         results.L_B = area_num_B / total_time_B;
335     else
336         results.P_B = NaN(1, K + 1);
337         results.L_B = NaN;
338     endif
339
340     if arrival_attempts_B > 0
341         results.P_rej_B = rejections_B / arrival_attempts_B;
342     else
343         results.P_rej_B = NaN;
344     endif
345
346     if served_customers_B > 0
347         results.W_B = sojourn_sum_B / served_customers_B;
348     else
349         results.W_B = NaN;
350     endif
351
352     % Curves
353     results.sample_steps = sample_steps(1:sample_idx);
354     results.L_evolution_all = L_evolution_all(1:sample_idx);
355
356     if sample_idx_B > 0
357         results.sample_steps_B = sample_steps_B(1:sample_idx_B);
358         results.L_evolution_B = L_evolution_B(1:sample_idx_B);
359     else
360         results.sample_steps_B = [];
361         results.L_evolution_B = [];
362     endif
363 endfunction
364
365
366 function print_results(results)
367     fprintf('lambda = %g, mu = %g, K = %d, T = %d, B = %d\n', ...
368         results.lambda, results.mu, results.K, results.T, results.B);
369     fprintf('Runtime = %.4f sec\n\n', results.runtime_sec);
370
371     fprintf('--- Using all transitions ---\n');
372     fprintf('Mean number in system L      = %.10f\n', results.L);
373     fprintf('Rejection probability P_rej    = %.10f\n', results.P_rej);
374     fprintf('Mean time in system W      = %.10f\n', results.W);

```

```

375     fprintf('State probabilities P          = ');
376     fprintf(' %.8f', results.P);
377     fprintf(' ]\n\n');
378
379     fprintf('--- Ignoring first B transitions ---\n');
380     fprintf('Mean number in system L_B      = %.10f\n', results.L_B);
381     fprintf('Rejection probability P_rej_B = %.10f\n', results.P_rej_B);
382     fprintf('Mean time in system W_B          = %.10f\n', results.W_B);
383     fprintf('State probabilities P_B          = ');
384     fprintf(' %.8f', results.P_B);
385     fprintf(' ]\n\n');
386
387     fprintf('Relative difference in L          = %.6f %%\n', ...
388           100 * abs(results.L_B - results.L) / max(abs(results.L), eps));
389     fprintf('Relative difference in P_rej      = %.6f %%\n', ...
390           100 * abs(results.P_rej_B - results.P_rej) / max(abs(results.P_rej), eps));
391     fprintf('Relative difference in W          = %.6f %%\n\n', ...
392           100 * abs(results.W_B - results.W) / max(abs(results.W), eps));
393 endfunction
394
395
396 function safe_legend_outside(hleg)
397     % Places legend outside the axes when supported by the Octave version.
398     % Falls back to northeast if needed.
399     try
400         set(hleg, 'location', 'eastoutside');
401     catch
402         try
403             legend(hleg, 'location', 'eastoutside');
404         catch
405             try
406                 set(hleg, 'location', 'northeast');
407             catch
408                 % do nothing
409             end_try_catch
410         end_try_catch
411     end_try_catch
412 endfunction

```